



**VASANTDADA PATIL PRATISHTHAN'S COLLEGE OF
ENGINEERING AND VISUAL ARTS**

Department of Humanities and Basic Science

**LAB MANUAL
OF
Data Structure Lab**

Code: PCL2011

**Class: First Year Engineering
Semester: II**

(NEP 2020 w.e.f. Academic Year 2024-25)

Lab Incharge

Dr.Akhilesh K. Pandey

H.O.D

Dr.A.Y.Shete



College Vision

To provide an environment to educate, encourage and explore students by facilitating innovative research, entrepreneurship, opportunities and employability to achieve social and professional goals.

College Mission

- To foster entrepreneurship & strengthen industry institute interaction to enhance career opportunities for the employability of students.
- To encourage collaborations with industries and academic institutes in terms of projects & internships by creating area for Research and Development.
- To build up appropriate moral and ethical skills and to promote holistic development of students through various academic, social and cultural activities



Lab Objectives:

The course aims to

1. Learn about the purpose and importance of data structures, as well as their principles.
2. Understand linear and nonlinear data structures, as well as their implementation.
3. Analyze data structures, such as stacks and queues.
4. Study the terminologies, types, and various operations in linked lists.
5. Discover the principles of Tree, including its operations and uses.
6. Investigate the real-time uses of different data structures.



Lab Outcomes:

After successful completion of the course students will be able to

1. Classify and apply linear and non-linear data structure concepts to real-world problem solving, as well as performing operations such as insertion, deletion, and traversal.
2. Explore data structures like Stacks, learn about their operations, and apply them to solve issues in a variety of domains.
3. Examine queue data structures and apply them to use in diverse real-world applications.
4. Apply the concept of linked lists to evaluate problems in a variety of applications.
5. Analyze and apply the concepts of Trees and their applications in real life problem solving
6. Demonstrate the ability to analyze, construct, implement, and use data structures to solve real-world problems and evaluate their effectiveness.

Prerequisite: Fundamentals of C programming and its concepts like Functions, Recursion, Arrays, Structures and Pointers.



Sr. No.	Module	Detailed Content	LO Mapping
0	Prerequisite	Constructs of C like Functions, Recursion, Arrays, Structures and Pointers.	
I	Introduction	Overview of Data Structure, Elementary Data Structure Organization, Classification of Data Structures, Operations on Data Structures and Abstract Data Type, recursion.	LO 1
II	Stack	Introduction to Stacks, Array representation of Stacks, Operations on a Stack.	LO 2
III	Queue	Introduction to Queues, Array representation of Queues, Types of Queues, Operations on Queue, Applications of Queues.	LO 3
IV	Linked list	Basics of Linked list, ADT Operations Singly Linked	LO 4



**VASANTDADA PATIL PRATISHTHAN'S COLLEGE OF
ENGINEERING AND VISUAL ARTS**

Department of Humanities and Basic Science

		Lists, Circular Linked Lists, Doubly Linked Lists, Linked representation of Stacks, and Linked Representation of Queues.	
V	Tree	Basic Terminology, Types of Trees, Binary Tree traversal, Operations on Binary Search Trees.	LO 5
VI	Applications of Data Structures	Stack: Reversing a list/String, Implementing Parentheses Checker, Evaluation of Arithmetic Expressions, Tree: Evaluating the expressions using expression tree and implementation of Huffman Encoding.	LO 6



Sr. No.	Name of the Experiment
1	Implementation of Insertion and deletion in a specific position in an Array using Function.
2	Implementation of recursive program.
3	Array Implementation of Stack.
4	Array Implementation of Linear Queue.
5	Array Implementation of Circular Queue.
6	Implement Singly Linked List.
7	Implement Doubly Linked List.
8	Implementation of Double Ended Queue using Linked List.
9	Implementation of Stack using Linked list
10	Implementation of Binary Search Tree and its traversal methods.
11	Program to count Number of leaf nodes, find the biggest and smallest and height of the tree.
12	Implementation of Reversing a List using Stack.
13	Convert an Infix expression to Postfix expression using stack ADT.
14	Program to Evaluate Postfix Expression using Stack ADT.



Introduction to Data Structures

Data structures are ways to organize and store data in a computer so that it can be efficiently accessed and manipulated. Data structures provide a means to manage large amounts of data efficiently, allowing for operations like insertion, deletion, searching, and traversal. In computer science, data structures are fundamental to designing efficient algorithms and programs.

Hardware and Software Requirements

Hardware Requirements:

- Desktop Computer / Laptop Computer with minimum 1 GB RAM and 500 MB free disk space

Software Requirements:

- Linux Operating System with GCC Compiler (recommended) or
- Turbo C / GCC Compiler on Windows OS

About GCC (GNU Compiler Collection):



GCC is a free, open-source compiler released by the Free Software Foundation. It is a command-line based C compiler commonly included with most Linux distributions. If you are using Linux or UNIX, GCC is usually already available on your system.

Compiling a C Program in GCC:

1. To compile a C source file, use: `gcc filename.c`
2. This generates an executable file named `a.out` by default.
3. To run it: `./a.out`
4. Creating an Executable with a Custom Name: `gcc filename.c -o outputfile`
5. To run: `./outputfile`

Note: The `./` ensures the program is executed from the current directory.

Using Math Library Functions:

If your program uses mathematical functions like `sin()`, `sqrt()`, etc., include the math library using the `-lm` flag: `gcc filename.c -o outputfile -lm`



About Turbo C/C++ (Windows Environment):

1. Open Turbo C from Desktop/Programs.
2. Select File → New and save your file with a .c extension.
3. To compile the program: Compile → Compile
 - If successful, a “Success” message will appear.
 - Otherwise, errors will be displayed.
4. To run the program: Run → Run
 - The output screen will appear.

Recommended System Configuration:

- Processor: Intel Core 2 Duo or equivalent
- RAM: 2 GB or more
- Operating System: Linux (Ubuntu, Fedora, etc.) or Windows (XP, 7, 8, 10)
- Compiler: GCC (Linux) or Turbo C / GCC (Windows)

Note: The above configuration is recommended for smooth execution of the experiments. However, the experiments can be performed on lower configurations as well.



Experiment 1: Implementation of Insertion and Deletion in a Specific Position in an Array using Function

Aim:

Implement insertion and deletion operations in an array at a specific position using functions.

Objectives:

- Understand array operations
- Learn to implement insertion and deletion using functions
- Handle array indexing and shifting elements

Theory:

An array is a linear data structure that stores elements of the same data type in contiguous memory locations. Insertion involves shifting elements to make space for the new element, while deletion involves shifting elements to fill the gap.



C Program:

```
#include <stdio.h>

void insert(int arr[], int *n, int pos, int val) {
    // Shift elements to make space for new element
    for (int i = *n; i > pos; i--) {
        arr[i] = arr[i-1];
    }
    arr[pos] = val;
    (*n)++;
}

void delete(int arr[], int *n, int pos) {
    // Shift elements to fill the gap
    for (int i = pos; i < *n - 1; i++) {
        arr[i] = arr[i+1];
    }
    (*n)--;
}

int main() {
    int arr[100], n = 5, pos, val;
    printf("Enter 5 array elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    printf("Enter position and value to insert: ");
```



```
scanf("%d %d", &pos, &val);  
insert(arr, &n, pos, val);  
printf("Array after insertion: ");  
for (int i = 0; i < n; i++) printf("%d ", arr[i]);  
printf("\nEnter position to delete: ");  
scanf("%d", &pos);  
delete(arr, &n, pos);  
printf("Array after deletion: ");  
for (int i = 0; i < n; i++) printf("%d ", arr[i]);  
return 0;  
}
```

Tentative Input/Output:

- Input: 10 20 30 40 50, position 2, value 25, delete position 3
- Output: Array after insertion: 10 20 25 30 40 50, Array after deletion: 10 20 25 40 50

Result:

The program successfully implemented insertion and deletion operations in an array at a specific position using functions.

Conclusion:

This experiment helped understand array operations, insertion, and deletion techniques, and how to handle array indexing and shifting elements.



Viva Questions:

1. What is an array?
2. How do you insert an element at a specific position in an array?
3. How do you delete an element from a specific position in an array?

Troubleshooting:

- Make sure to check for array bounds when inserting or deleting elements.
- Don't forget to update the array size after insertion or deletion.



Experiment 2: Implementation of Recursive Program

Aim:

Understand and implement recursive programs in C.

Objectives:

- Learn recursion concepts
- Implement recursive functions
- Understand base cases and recursive calls

Theory:

Recursion is a programming technique where a function calls itself repeatedly until a base condition is met. Recursive programs can solve problems by breaking them into smaller sub-problems.

C Program:

```
#include <stdio.h>
```

```
int factorial(int n) {
```

```
    // Base case: factorial of 0 is 1
```

```
    if (n == 0) return 1;
```

```
    // Recursive call:  $n! = n * (n-1)!$ 
```

```
    else return n * factorial(n-1);
```



```
}  
  
int main() {  
    int num;  
  
    printf("Enter a number: ");  
  
    scanf("%d", &num);  
  
    printf("Factorial: %d\n", factorial(num));  
  
    return 0;  
}
```

Tentative Input/Output:

- Input: 5
- Output: Factorial: 120

Result:

The program successfully implemented a recursive function to calculate the factorial of a number.

Conclusion:

This experiment helped understand recursion concepts and how to implement recursive functions.



Viva Questions:

1. What is recursion?
2. How do you implement a recursive function?
3. What is a base case in recursion?

Troubleshooting:

- Make sure to define a base case to stop the recursion.
- Check for stack overflow errors due to infinite recursion.



Experiment 3: Array Implementation of Stack

Aim:

Implement a stack using an array in C.

Objectives:

- Understand stack operations (push, pop)
- Implement stack using an array
- Handle stack overflow and underflow conditions

Theory:

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. Array implementation of a stack involves using an array to store elements and keeping track of the top index.

C Program:

```
#include <stdio.h>

#define MAX 100

int stack[MAX], top = -1;

void push(int val) {
    // Check for stack overflow
    if (top == MAX - 1) printf("Stack Overflow\n");
    else stack[++top] = val;
```



```
}
```

```
int pop() {
```

```
    // Check for stack underflow
```

```
    if (top == -1) {
```

```
        printf("Stack Underflow\n");
```

```
        return -1;
```

```
    } else return stack[top--];
```

```
}
```

```
int main() {
```

```
    int choice, val;
```

```
    while (1) {
```

```
        printf("1. Push 2. Pop 3. Exit: ");
```

```
        scanf("%d", &choice);
```

```
        if (choice == 1) {
```

```
            printf("Enter value: ");
```

```
            scanf("%d", &val);
```

```
            push(val);
```

```
        } else if (choice == 2) {
```

```
            printf("Popped: %d\n", pop());
```

```
        } else break;
```



```
}
```

```
return 0;
```

```
}
```

Tentative Input/Output:

- Input: 1, value 10, 1, value 20, 2
- Output: Popped: 20

Result:

The program successfully implemented a stack using an array and handled stack overflow and underflow conditions.

Conclusion:

This experiment helped understand stack operations and how to implement a stack using an array.

Viva Questions:

1. What is a stack?
2. How do you implement a stack using an array?
3. What are the conditions for stack overflow and underflow?

Troubleshooting:

- Make sure to check for stack overflow and underflow conditions.
- Don't forget to update the top index after push and pop operations.



Experiment 4: Array Implementation of Linear Queue

Aim:

Implement a linear queue using an array in C.

Objectives:

- Understand queue operations (enqueue, dequeue)
- Implement queue using an array
- Handle queue overflow and underflow conditions

Theory:

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. Array implementation of a queue involves using an array to store elements and keeping track of front and rear indices.

C Program:

```
#include <stdio.h>

#define MAX 100

int queue[MAX], front = -1, rear = -1;

void enqueue(int val) {
    // Check for queue overflow
    if (rear == MAX - 1) printf("Queue Overflow\n");
    else {
```



```
    if (front == -1) front = 0;

    queue[++rear] = val;

}

}

int dequeue() {

    // Check for queue underflow

    if (front == -1 || front > rear) {

        printf("Queue Underflow\n");

        return -1;

    } else return queue[front++];

}

int main() {

    int choice, val;

    while (1) {

        printf("1. Enqueue 2. Dequeue 3. Exit: ");

        scanf("%d", &choice);

        if (choice == 1) {

            printf("Enter value: ");

            scanf("%d", &val);

            enqueue(val);

        } else if (choice == 2) {

            printf("Dequeued: %d\n", dequeue());

        } else break;
```



}

return 0;

}

Tentative Input/Output:

- Input: 1, value 10, 1, value 20, 2
- Output: Dequeued: 10

Result:

The program successfully implemented a linear queue using an array and handled queue overflow and underflow conditions.

Conclusion:

This experiment helped understand queue operations and how to implement a queue using an array.

Viva Questions:

1. What is a queue?
2. How do you implement a queue using an array?
3. What are the conditions for queue overflow and underflow?

Troubleshooting:

- Make sure to check for queue overflow and underflow conditions.
- Don't forget to update the front and rear indices after enqueue and dequeue operations.



Experiment 5: Array Implementation of Circular Queue

Aim:

Implement a circular queue using an array in C.

Objectives:

- Understand circular queue operations
- Implement circular queue using an array
- Handle wrap-around conditions

Theory:

A circular queue is a type of queue where the last element is connected to the first element, forming a circle. Array implementation involves using modulo arithmetic to handle wrap-around.

C Program:

```
#include <stdio.h>

#define MAX 100

int queue[MAX], front = -1, rear = -1;

void enqueue(int val) {
    // Check for queue overflow
    if ((rear + 1) % MAX == front) printf("Queue Overflow\n");
    else {
        if (front == -1) front = 0;
        rear = (rear + 1) % MAX;
```




```
queue[rear] = val;

}

}

int dequeue() {
    // Check for queue underflow
    if (front == -1) {
        printf("Queue Underflow\n");
        return -1;
    } else {
        int temp = queue[front];
        if (front == rear) front = rear = -1;
        else front = (front + 1) % MAX;
        return temp;
    }
}

int main() {
    int choice, val;
    while (1) {
        printf("1. Enqueue 2. Dequeue 3. Exit: ");
        scanf("%d", &choice);
        if (choice == 1) {
            printf("Enter value: ");
            scanf("%d", &val);
```



```
enqueue(val);  
} else if (choice == 2) {  
    printf("Dequeued: %d\n", dequeue());  
} else break;  
}  
return 0;  
}
```

Tentative Input/Output:

- Input: 1, value 10, 1, value 20, 2
- Output: Dequeued: 10

Result:

The program successfully implemented a circular queue using an array and handled wrap-around conditions.

Conclusion:

This experiment helped understand circular queue operations and how to implement a circular queue using an array.



Viva Questions:

1. What is a circular queue?
2. How do you implement a circular queue using an array?
3. What is the purpose of using modulo arithmetic in a circular queue?

Troubleshooting:

- Make sure to check for queue overflow and underflow conditions.
- Don't forget to update the front and rear indices correctly in enqueue and dequeue operations.



Experiment 6: Implement Singly Linked List

Aim:

Implement a singly linked list in C.

Objectives:

- Understand linked list concepts
- Implement insertion, deletion, and traversal operations
- Manage dynamic memory allocation

Theory:

A singly linked list is a dynamic data structure where each element (node) points to the next node. It allows efficient insertion and deletion at any position.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *head = NULL;
void insert(int val) {
    // Create a new node and insert at the beginning
```



```
struct node *newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = val;
newnode->next = head;
head = newnode;
}

void delete(int val) {
    // Delete the node with the given value
    struct node *temp = head, *prev;
    if (temp != NULL && temp->data == val) {
        head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != val) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Value not found in the list\n");
        return;
    }
    prev->next = temp->next;
    free(temp);
}
```



}

```
void display() {
```

```
    // Traverse and display the linked list
```

```
    struct node *temp = head;
```

```
    while (temp != NULL) {
```

```
        printf("%d ", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
```

```
    insert(10);
```

```
    insert(20);
```

```
    insert(30);
```

```
    printf("List: ");
```

```
    display(); // Output: 30 20 10
```

```
    delete(20);
```

```
    printf("List after deletion: ");
```

```
    display(); // Output: 30 10
```

```
    return 0;
```

```
}
```



Tentative Input/Output:

- Insert values: 10, 20, 30
- List: 30 20 10
- Delete value: 20
- List after deletion: 30 10

Result:

The program successfully implemented a singly linked list with insertion, deletion, and traversal operations.

Conclusion:

This experiment helped understand linked list concepts and how to implement a singly linked list in C.

Viva Questions:

1. What is a singly linked list?
2. How do you insert a node at the beginning of a linked list?
3. How do you delete a node with a given value from a linked list?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the linked list.
- Don't forget to free the memory allocated for deleted nodes.



Experiment 7: Implement Doubly Linked List

Aim:

Implement a doubly linked list in C.

Objectives:

- Understand doubly linked list concepts
- Implement insertion, deletion, and traversal operations
- Manage dynamic memory allocation

Theory:

A doubly linked list is a dynamic data structure where each element (node) points to both the previous and next nodes. It allows efficient insertion and deletion at any position and traversal in both directions.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *prev, *next;
};
struct node *head = NULL;
void insert(int val) {
    // Create a new node and insert at the beginning
```




```
struct node *newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = val;
newnode->prev = NULL;
newnode->next = head;
if (head != NULL) head->prev = newnode;
head = newnode;
}

void delete(int val) {
    // Delete the node with the given value
    struct node *temp = head;
    while (temp != NULL && temp->data != val) temp = temp->next;
    if (temp == NULL) return;
    if (temp->prev != NULL) temp->prev->next = temp->next;
    else head = temp->next;
    if (temp->next != NULL) temp->next->prev = temp->prev;
    free(temp);
}

void display() {
    // Traverse and display the doubly linked list
    struct node *temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```



```
}  
printf("\n");  
}  
int main() {  
    int choice, val;  
    while (1) {  
        printf("1. Insert 2. Delete 3. Display 4. Exit: ");  
        scanf("%d", &choice);  
        if (choice == 1) {  
            printf("Enter value: ");  
            scanf("%d", &val);  
            insert(val);  
        } else if (choice == 2) {  
            printf("Enter value to delete: ");  
            scanf("%d", &val);  
            delete(val);  
        } else if (choice == 3) display();  
        else break;  
    }  
    return 0;  
}
```



Tentative Input/Output:

- Input: 1, value 10, 1, value 20, 3
- Output: 20 10
- Input: 2, value 10, 3
- Output: 20

Result:

The program successfully implemented a doubly linked list with insertion, deletion, and traversal operations.

Conclusion:

This experiment helped understand doubly linked list concepts and how to implement a doubly linked list in C.

Viva Questions:

1. What is a doubly linked list?
2. How do you insert a node at the beginning of a doubly linked list?
3. How do you delete a node with a given value from a doubly linked list?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the doubly linked list.
- Don't forget to update the prev and next pointers of adjacent nodes when inserting or deleting a node.
- Don't forget to free the memory allocated for deleted nodes.



Experiment 8: Implementation of Double Ended Queue using Linked List

Aim:

Implement a double-ended queue (deque) using a linked list in C.

Objectives:

- Understand deque operations (insertFront, insertRear, deleteFront, deleteRear)
- Implement deque using a linked list
- Manage dynamic memory allocation

Theory:

A double-ended queue (deque) is a data structure that allows insertion and deletion at both ends. It can be implemented using a linked list with pointers to both the front and rear nodes.

C Program:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
```



```
struct node *front = NULL, *rear = NULL;

void insertFront(int val) {
    // Insert a node at the front of the deque
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = front;
    front = newnode;
    if (rear == NULL) rear = front;
}

void insertRear(int val) {
    // Insert a node at the rear of the deque
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = NULL;
    if (rear == NULL) front = rear = newnode;
    else rear->next = newnode, rear = newnode;
}

int deleteFront() {
    // Delete a node from the front of the deque
    if (front == NULL) {
        printf("Deque Underflow\n");
        return -1;
    }
}
```



```
}  
struct node *temp = front;  
int val = temp->data;  
front = front->next;  
if (front == NULL) rear = NULL;  
free(temp);  
return val;  
}  
int deleteRear() {  
    // Delete a node from the rear of the deque  
    if (rear == NULL) {  
        printf("Deque Underflow\n");  
        return -1;  
    }  
    struct node *temp = front;  
    int val = rear->data;  
    if (front == rear) front = rear = NULL;  
    else {  
        while (temp->next != rear) temp = temp->next;  
        rear = temp;  
        rear->next = NULL;  
    }  
    free(temp->next);
```



```
return val;

}

int main() {
    int choice, val;
    while (1) {
        printf("1. Insert Front 2. Insert Rear 3. Delete Front 4. Delete
Rear 5. Exit: ");
        scanf("%d", &choice);
        if (choice == 1) {
            printf("Enter value: ");
            scanf("%d", &val);
            insertFront(val);
        } else if (choice == 2) {
            printf("Enter value: ");
            scanf("%d", &val);
            insertRear(val);
        } else if (choice == 3) printf("Deleted: %d\n", deleteFront());
        else if (choice == 4) printf("Deleted: %d\n", deleteRear());
        else break;
    }
    return 0;
}
```



Tentative Input/Output:

- Input: 1, value 10, 2, value 20, 3
- Output: Deleted: 10
- Input: 4
- Output: Deleted: 20

Result:

The program successfully implemented a double-ended queue using a linked list.

Conclusion:

This experiment helped understand deque operations and how to implement a deque using a linked list.

Viva Questions:

1. What is a double-ended queue (deque)?
2. How do you insert a node at the front of a deque?
3. How do you delete a node from the rear of a deque?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the deque.
- Don't forget to update the front and rear pointers correctly in insertion and deletion operations.
- Don't forget to free the memory allocated for deleted nodes



Experiment 9: Implementation of Stack using Linked List

Aim:

Implement a stack using a linked list in C.

Objectives:

- Understand stack operations using a linked list
- Implement push and pop operations
- Manage dynamic memory allocation

Theory:

A stack can be implemented using a linked list where each node represents an element. The top of the stack is the head of the list.

C Program:

```
#include <stdio.h>

#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *top = NULL;

void push(int val) {
    // Push an element onto the stack
```



```
struct node *newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = val;
newnode->next = top;
top = newnode;
}

int pop() {
    // Pop an element from the stack
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct node *temp = top;
    int val = temp->data;
    top = top->next;
    free(temp);
    return val;
}

int main() {
    int choice, val;
    while (1) {
        printf("1. Push 2. Pop 3. Exit: ");
        scanf("%d", &choice);
        if (choice == 1) {
```



```
printf("Enter value: ");  
scanf("%d", &val);  
push(val);  
} else if (choice == 2) printf("Popped: %d\n", pop());  
else break;  
}  
return 0;  
}
```

Tentative Input/Output:

- Input: 1, value 10, 1, value 20, 2
- Output: Popped: 20

Result:

The program successfully implemented a stack using a linked list.

Conclusion:

This experiment helped understand stack operations and how to implement a stack using a linked list.

Viva Questions:

1. What is a stack?
2. How do you implement a stack using a linked list?
3. What are the advantages of implementing a stack using a linked list?



Troubleshooting:

- Make sure to check for NULL pointers when popping an element from the stack.
- Don't forget to free the memory allocated for popped nodes.
- Don't forget to update the top pointer correctly in push and pop operations.



Experiment 10: Implementation of Binary Search Tree and its Traversal Methods

Aim:

Implement a binary search tree (BST) and its traversal methods in C.

Objectives:

- Understand BST concepts and operations
- Implement insertion, inorder, preorder, and postorder traversals
- Manage dynamic memory allocation

Theory:

A binary search tree is a binary tree where each node has a value greater than all values in its left subtree and less than all values in its right subtree.

Traversal methods visit nodes in specific orders: inorder (left-root-right), preorder (root-left-right), and postorder (left-right-root). BSTs enable efficient searching, insertion, and deletion operations.



C Program:

```
#include <stdio.h>

#include <stdlib.h>

struct node {
int data;
struct node *left, *right;
};

struct node* insert(struct node *root, int val) {
// Insert a node into the BST
if (root == NULL) {
root = (struct node*)malloc(sizeof(struct node));
root->data = val;
root->left = root->right = NULL;
} else if (val < root->data) root->left = insert(root-
>left, val);
else root->right = insert(root->right, val);
return root;
}

void inorder(struct node *root) {
// Inorder traversal (left-root-right)
if (root != NULL) {
inorder(root->left);
printf("%d ", root->data);
```



```
inorder(root->right);
}
}

void preorder(struct node *root) {
// Preorder traversal (root-left-right)
if (root != NULL) {
printf("%d ", root->data);
preorder(root->left);
preorder(root->right);
}
}

void postorder(struct node *root) {
// Postorder traversal (left-right-root)
if (root != NULL) {
postorder(root->left);
postorder(root->right);
printf("%d ", root->data);
}
}

int main() {
struct node *root = NULL;
int choice, val;
while (1) {
```



```
printf("1. Insert 2. Inorder 3. Preorder 4. Postorder  
5. Exit: ");  
scanf("%d", &choice);  
if (choice == 1) {  
    printf("Enter value: ");  
    scanf("%d", &val);  
    root = insert(root, val);  
} else if (choice == 2) {  
    printf("Inorder traversal: ");  
    inorder(root);  
    printf("\n");  
} else if (choice == 3) {  
    printf("Preorder traversal: ");  
    preorder(root);  
    printf("\n");  
} else if (choice == 4) {  
    printf("Postorder traversal: ");  
    postorder(root);  
    printf("\n");  
} else break;  
}  
return 0;
```




}

Tentative Input/Output:

- Input: 1, value 50, 1, value 30, 1, value 20, 1, value 40, 1, value 70, 1, value 60, 1, value 80
- Input: 2
- Output: Inorder traversal: 20 30 40 50 60 70 80
- Input: 3
- Output: Preorder traversal: 50 30 20 40 70 60 80
- Input: 4
- Output: Postorder traversal: 20 40 30 60 80 70 50

Result:

The program successfully implemented a binary search tree with insertion and three traversal methods.

Conclusion:

This experiment helped understand BST concepts and how to implement a BST with traversal methods.



Viva Questions:

1. What is a binary search tree?
2. What are the different traversal methods for a BST?
3. How do you insert a node into a BST?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the BST.
- Don't forget to update the left and right child pointers correctly in insertion operations.
- Don't forget to free the memory allocated for deleted nodes.



Experiment 11: Count Leaf Nodes in a Binary Tree

Aim:

Implement a program to count the number of leaf nodes in a binary tree.

Objectives:

- Understand binary tree concepts
- Implement a recursive function to count leaf nodes
- Manage tree traversal

Theory:

A leaf node in a binary tree is a node with no children (both left and right subtrees are NULL). Counting leaf nodes involves traversing the tree and identifying nodes that meet this condition. Leaf nodes represent the endpoints of the tree structure.

C Program:

```
#include <stdio.h>

#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};
```



```
struct node* createNode(int val) {  
    // Create a new node with the given value  
    struct node *newnode = (struct node*)malloc(sizeof(struct node));  
    newnode->data = val;  
    newnode->left = newnode->right = NULL;  
    return newnode;  
}  
  
int countLeafNodes(struct node *root) {  
    // Count the number of leaf nodes in the binary tree  
    if (root == NULL) return 0;  
    if (root->left == NULL && root->right == NULL) return 1;  
    return countLeafNodes(root->left) + countLeafNodes(root->right);  
}  
  
int main() {  
    struct node *root = createNode(1);  
    root->left = createNode(2);  
    root->right = createNode(3);  
    root->left->left = createNode(4);  
    root->left->right = createNode(5);  
    printf("Number of leaf nodes: %d\n", countLeafNodes(root));  
}
```



```
return 0;  
  
}
```

Tentative Input/Output:

- Tree structure: 1

/

2 3

/

4 5

- Output: Number of leaf nodes: 3

Result:

The program successfully counted the number of leaf nodes in the binary tree.

Conclusion:

This experiment helped understand binary tree concepts and how to implement a recursive function to count leaf nodes.



Viva Questions:

1. What is a leaf node in a binary tree?
2. How do you count the number of leaf nodes in a binary tree?
3. What is the time complexity of the countLeafNodes function?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the binary tree.
- Don't forget to handle the base case where the tree is empty (root is NULL).
- Don't forget to recursively count leaf nodes in the left and right subtrees.



Experiment 11: Count Leaf Nodes in a Binary Tree

Aim:

Implement a program to count the number of leaf nodes in a binary tree.

Objectives:

- Understand binary tree concepts
- Implement a recursive function to count leaf nodes
- Manage tree traversal

Theory:

A leaf node in a binary tree is a node with no children (both left and right subtrees are NULL). Counting leaf nodes involves traversing the tree and identifying nodes that meet this condition. Leaf nodes represent the endpoints of the tree structure.

C Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *left, *right;
```



```
};
```

```
struct node* createNode(int val) {
```

```
    // Create a new node with the given value
```

```
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
```

```
    newnode->data = val;
```

```
    newnode->left = newnode->right = NULL;
```

```
    return newnode;
```

```
}
```

```
int countLeafNodes(struct node *root) {
```

```
    // Count the number of leaf nodes in the binary tree
```

```
    if (root == NULL) return 0;
```

```
    if (root->left == NULL && root->right == NULL) return 1;
```

```
    return countLeafNodes(root->left) + countLeafNodes(root->right);
```

```
}
```

```
int main() {
```

```
    struct node *root = createNode(1);
```

```
    root->left = createNode(2);
```

```
    root->right = createNode(3);
```

```
    root->left->left = createNode(4);
```




```
root->left->right = createNode(5);  
  
printf("Number of leaf nodes: %d\n", countLeafNodes(root));  
  
return 0;  
  
}
```

Tentative Input/Output:

- Tree structure: 1

/

2 3

/

4 5

- Output: Number of leaf nodes: 3

Result:

The program successfully counted the number of leaf nodes in the binary tree.

Conclusion:

This experiment helped understand binary tree concepts and how to implement a recursive function to count leaf nodes.



Viva Questions:

1. What is a leaf node in a binary tree?
2. How do you count the number of leaf nodes in a binary tree?
3. What is the time complexity of the countLeafNodes function?

Troubleshooting:

- Make sure to check for NULL pointers when traversing the binary tree.
- Don't forget to handle the base case where the tree is empty (root is NULL).
- Don't forget to recursively count leaf nodes in the left and right subtrees.



Experiment 12: Implementation of Reversing a List using Stack

Aim:

Implement a program to reverse a list using a stack in C.

Objectives:

- Understand stack concepts and operations
- Implement a stack using an array
- Reverse a list using the stack

Theory:

A stack is a Last-In-First-Out (LIFO) data structure that can be used to reverse a list. The idea is to push all elements of the list onto the stack and then pop them off the stack, which reverses the order of the elements.

C Program:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

int stack[MAX];

int top = -1;
```



```
void push(int val) {  
    // Push an element onto the stack  
    if (top == MAX - 1) {  
        printf("Stack Overflow\n");  
        return;  
    }  
    stack[++top] = val;  
}  
  
int pop() {  
    // Pop an element from the stack  
    if (top == -1) {  
        printf("Stack Underflow\n");  
        return -1;  
    }  
    return stack[top--];  
}  
  
void reverseList(int arr[], int n) {  
    // Reverse the list using the stack  
    for (int i = 0; i < n; i++) push(arr[i]);
```



```
for (int i = 0; i < n; i++) arr[i] = pop();

}

void printList(int arr[], int n) {

    // Print the list

    for (int i = 0; i < n; i++) printf("%d ", arr[i]);

    printf("\n");

}

int main() {

    int arr[] = {1, 2, 3, 4, 5};

    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original List: ");

    printList(arr, n);

    reverseList(arr, n);

    printf("Reversed List: ");

    printList(arr, n);

    return 0;

}
```



Tentative Input/Output:

- Original List: 1 2 3 4 5
- Reversed List: 5 4 3 2 1

Result:

The program successfully reversed the list using a stack.

Conclusion:

This experiment helped understand stack concepts and how to use a stack to reverse a list.

Viva Questions:

1. What is a stack?
2. How do you reverse a list using a stack?
3. What are the advantages of using a stack to reverse a list?

Troubleshooting:

- Make sure to check for stack overflow and underflow conditions.
- Don't forget to update the top index correctly in push and pop operations.
- Don't forget to handle the case where the list is empty.



Experiment 13: Convert Infix to Postfix using Stack ADT

Aim:

Implement a program to convert an infix expression to postfix notation using a stack.

Objectives:

- Understand infix and postfix notations
- Implement conversion algorithm using a stack
- Manage operator precedence

Theory:

Infix notation (e.g., $A+B$) is commonly used in arithmetic expressions. Postfix notation (e.g., $AB+$) places operators after operands. Conversion from infix to postfix uses a stack to manage operators based on precedence rules. Postfix notation eliminates the need for parentheses and is easier for computers to evaluate.

C Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
char stack[MAX];
int top = -1;
```



```
void push(char c) {  
    // Push a character onto the stack  
    if (top == MAX - 1) printf("Stack Overflow\n");  
    else stack[++top] = c;  
}  
  
char pop() {  
    // Pop a character from the stack  
    if (top == -1) return -1;  
    else return stack[top--];  
}  
  
int precedence(char c) {  
    // Return the precedence of the operator  
    if (c == '+' || c == '-') return 1;  
    if (c == '*' || c == '/') return 2;  
    return 0;  
}  
  
void infixToPostfix(char *infix, char *postfix) {  
    // Convert infix to postfix using a stack  
    int j = 0;  
    for (int i = 0; i < strlen(infix); i++) {  
        char c = infix[i];  
        if (isalnum(c)) postfix[j++] = c;  
        else if (c == '(') push(c);
```




```
else if (c == ')') {
    while (stack[top] != '(') postfix[j++] = pop();
    pop(); // remove '('
} else {
    while (top != -1 && precedence(stack[top]) >= precedence(c))
postfix[j++] = pop();
    push(c);
}
}
while (top != -1) postfix[j++] = pop();
postfix[j] = '\0';
}

int main() {
    char infix[100], postfix[100];
    printf("Enter infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```



Tentative Input/Output:

- Input: $A+B*C$
- Output: Postfix expression: $ABC*+$
- Input: $(A+B)*C$
- Output: Postfix expression: $AB+C*$

Result:

The program successfully converted the infix expression to postfix notation using a stack.

Conclusion:

This experiment helped understand infix and postfix notations and how to implement a conversion algorithm using a stack.

Viva Questions:

1. What is infix notation?
2. What is postfix notation?
3. How do you convert an infix expression to postfix using a stack?

Troubleshooting:

- Make sure to check for stack overflow and underflow conditions.
- Don't forget to handle parentheses correctly.
- Don't forget to pop all remaining operators from the stack at the end.



Experiment 14: Program to Evaluate Postfix Expression using Stack ADT

Aim:

Implement a program to evaluate a postfix expression using a stack Abstract Data Type (ADT).

Objectives:

- Understand postfix notation
- Implement evaluation algorithm using a stack
- Manage stack operations

Theory:

A postfix expression is a notation where operators follow their operands (e.g., AB+). Evaluating a postfix expression involves using a stack to store operands and apply operators.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX 100
int stack[MAX];
int top = -1;
```



```
void push(int val) {
    // Push a value onto the stack
    if (top == MAX - 1) printf("Stack Overflow\n");
    else stack[++top] = val;
}

int pop() {
    // Pop a value from the stack
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    } else return stack[top--];
}

int evaluatePostfix(char *exp) {
    // Evaluate the postfix expression using a stack
    int op1, op2, result;
    for (int i = 0; exp[i]; i++) {
        if (isdigit(exp[i])) push(exp[i] - '0');
        else {
            op2 = pop();
            op1 = pop();
            switch (exp[i]) {
                case '+': push(op1 + op2); break;
                case '-': push(op1 - op2); break;
```



```
        case '*': push(op1 * op2); break;
        case '/': push(op1 / op2); break;
    }
}
}
return pop();
}

int main() {
    char exp[100];
    printf("Enter postfix expression: ");
    scanf("%s", exp);
    printf("Result: %d\n", evaluatePostfix(exp));
    return 0;
}
```

Tentative Input/Output:

- Input: 23+5*
- Output: Result: 25

Result:

The program successfully evaluated a postfix expression using a stack.

Conclusion:

This experiment helped understand postfix notation and how to evaluate expressions using a stack.



Viva Questions:

1. What is postfix notation?
2. How do you evaluate a postfix expression using a stack?
3. What are the advantages of postfix notation?

Troubleshooting:

- Make sure to handle stack overflow and underflow conditions.
- Check for valid postfix expressions.
- Don't forget to pop the final result from the stack.